

Требования (Спецификация)

Что такое требования?

TЗ vs Спека vs Бриф

- ❖ Спецификация программного обеспечения позволяет зафиксировать детальные требования к разработке, указать роли и ответственности сторон, сроки, стоимость реализации, ограничения.
- ❖ ТЗ содержит инструкции и требования от заказчика к исполнителю проекта, детализирует технические аспекты разработки, такие как выбор языка программирования, архитектура системы, требования к аппаратному и программному обеспечению, сроки выполнения проекта и прочие важные детали, помогает определить порядок выполнения работ, установить сроки и распределить ответственность между участниками проекта
- ❖ Бриф – это опросник. Бриф отправляется клиентам, чтобы выяснить общие бизнес-требования, предполагаемый функционал и пожелания.

Требования (Спецификация)

- ✦ Наиболее точное определение:

Требования это спецификация того, что должно быть реализовано. В них описано поведение системы, свойства системы или ее атрибуты. Они могут служить ограничениями в процессе разработки системы.*

- ✦ Наиболее простое определение:

Условия или возможности, которым должна соответствовать система для удовлетворения нужд пользователей и заказчиков

Анализ требований

Процесс проверки, оценки и улучшения спеки перед началом разработки.

- Включает изучение, уточнение, классификацию и приоритезацию требований.
- Проверяется полнота, согласованность, реализуемость, однозначность и тестируемость требований.
- Выявляются противоречия, дубли и пробелы.
- Формируется модель требований (диаграммы, сценарии, прототипы).

Уровни требований

- > Бизнес-требования
- > Пользовательские требования
- > Функциональные требования

Типы требований

- Бизнес-правило
- Внешнее требование к интерфейсу
- Функциональное требование
- Нефункциональное требование
 - Атрибут качества

Описание требований

Требования должны быть **однозначными, полными, проверяемыми, реализуемыми и согласованными**.

Формулировки должны быть максимально точными, избегать двусмысленностей и субъективных оценок ("быстро", "удобно", "лучше").

Оформление:

- ✦ Использование шаблонов (например, IEEE 830, ГОСТ 34.602-89).
- ✦ Структурированное изложение (разделы, заголовки, списки).
- ✦ Таблицы, диаграммы UML для наглядности.
- ✦ Пронумерованные требования для удобства ссылок и отслеживания изменений.

Методологии и стандарты, регламентирующие работу с требованиями

- **IEEE 830-1998**: стандарт по описанию требований к программному обеспечению (SRS – Software Requirements Specification).
- **ISO/IEC/IEEE 29148**: международный стандарт, объединяющий лучшие практики по управлению требованиями.
- **BABOK (Business Analysis Body of Knowledge)**: свод знаний по бизнес-анализу, описывает процессы работы с требованиями.
- **ГОСТ 19.xxx** Единая система программной документации (ЕСПД)
- **ГОСТ 34.xxx** Стандарты информационных технологий, проектирования АС (комплекс стандартов, используется при производстве серийного продукта)

Базовые требования ко всем разделам

- ✓ Кратко, четко
- ✓ Без двусмысленных описаний
- ✓ Простота и «читабельность»
- ✓ Необходимость в DFD-диаграммах
- ✓ Степень детализации

ТЗ. Зачем нужно?

- **Фиксация пожеланий заказчика.** Все требования и ожидания клиента от продукта заносятся в документ, что гарантирует их реализацию.
- **Понимание бюджета, объёма и сроков работ.** Благодаря подробному описанию проекта команда может понять, какие задачи предстоит выполнить, сколько времени на это понадобится, и оценить стоимость работ.
- **Разрешение разногласий.** ТЗ, где зафиксированы содержание и требования к продукту, пригодится в случае споров между клиентом и исполнителем.

ТЗ. Этапы

- ✦ Заполнение брифа
- ✦ Сбор пожеланий заказчика
- ✦ Консультации с другими специалистами
- ✦ Оформление ТЗ
- ✦ Согласование
- ✦ Подписание ТЗ

Описание требований. Структура

- Введение
 - Назначение
 - Область действия
 - Определения и сокращения
 - Ссылки
 - Краткий обзор
- Общее описание
 - Взаимодействие продукта с другими продуктами и компонентами
 - Функции продукта (краткое описание)
 - Характеристики пользователя
 - Окружение
 - Допущения и зависимости
- Детальные требования
 - Требования к внешним интерфейсам
 - Интерфейсы пользователя
 - Интерфейсы аппаратного обеспечения
 - Интерфейсы программного обеспечения
 - Интерфейсы взаимодействия
 - Функциональные требования
 - Проектные ограничения и допущения (и ссылки на стандарты)
 - Нефункциональные требования (надежность, доступность, безопасность и пр.)
 - Другие требования
 - Критерии приемки
- Приложения
- Алфавитный указатель

Структура. Введение

- **Назначение/Цель:** четко указывает, почему разрабатывается программное обеспечение, для кого оно предназначено и какие цели преследует документ.
- **Область применения/объем:** определяет границы функциональности программного обеспечения, устанавливая четкие ожидания относительно того, что будет охватывать проект, а что нет. Обратите внимание на сроки или ограничения ресурсов, которые могут повлиять на проект.
- **Определения и сокращения:** Предоставляет глоссарий для стандартизации терминов и разъяснения технического языка, обеспечивая единообразное понимание среди заинтересованных сторон. Заметьте, что описание непонятого слова не может содержать другое непонятое слово.
- **Ссылки:** В данной секции мы пишем ссылки на литературу в которой можно найти основания использованных технологий и фактов. Можно вставлять ссылки на документы на которые вы ссылаетесь и тд. Ссылки и их описание должны быть максимально полными, чтобы в случае чего (линк умер просто) можно было нагуглить этот материал.
- **Краткий обзор**

Структура. Общее описание

- **Взаимодействие продукта с другими продуктами и компонентами:** Описывает, как программное обеспечение вписывается в более крупную систему или соотносится с существующими продуктами, включая зависимости, интерфейсы или интеграции.
- **Функции продукта:** Обобщает основные функции, предоставляя функциональный обзор, который объясняет основные возможности программного обеспечения, не вдаваясь в подробные детали. Тут желательно разместить DFD-диаграмму которая покажет общее взаимодействие системы.
- **Характеристики пользователей:** определяет различные типы конечных пользователей, отмечает конкретные потребности или ограничения пользователей для разработки дизайна, ориентированного на пользователя.
- **Окружение:** Как понятно из названия раздела тут мы описываем окружение в котором будет работать продукт. ОС, версии компиляторов, базы данных, сервера, софт, железо и другие вещи которые необходимы для работы вашего продукта.
- **Допущения и Зависимости**

Структура. Детальные требования

- **Требования к внешним интерфейсам** - описывают взаимодействие с пользователем и внешними системами. Здесь указывают форматы данных для интеграции, API, протоколы обмена информацией. Могут быть макеты.
- **Функциональные требования:** центральный раздел документа, описывают функциональность системы. Каждое требование формулируют чётко, измеримо, проверяемо.
- **Нефункциональные требования:** требования к производительности. Требования к качеству кода. Требования по безопасности. Требования по отказоустойчивости.
- **Проектные ограничения и допущения:** фиксируют рамки проекта. Ограничения – факторы, которые влияют на разработку: бюджет, сроки, законодательные требования, технические возможности инфраструктуры. Допущения – предположения, принятые за основу при планировании.
- **Критерии приёмки:** определяют условия успешного завершения разработки. По этим критериям заказчик оценивает готовность системы. Каждое требование из функционального раздела должно иметь свой критерий приёмки.
- **Приложения:** может содержать дополнительные материалы: схемы процессов, макеты интерфейса, примеры отчётов, спецификации форматов данных. Эти материалы помогают разработчикам лучше понять требования.

ФТ

- **Описывают, что должна делать система.** Они определяют конкретные действия и операции, которые система должна выполнять для удовлетворения потребностей пользователей.
- Некоторые особенности функциональных требований:
 - Отвечают на вопрос «что делает система».
 - Описывают поведение ПО в определённых условиях.
 - Часто документируются в виде списка с чётко сформулированными пользовательскими сценариями.

Пример формулировки: Система должна позволять пользователям регистрироваться, используя email и пароль.

Структура. ФТ

▪ **FRQ \ FRQ X**

- Называем фичу проекту и даем ей уникальный идентификатор. Например, server.html.editor. Уникальный идентификатор дается для того, чтобы потом где то в тикетах ваших не писать – «а вот та хреновина, которая позволяет редактировать html»

▪ **FRQ \ FRQ X \ Description and priority**

- Описываем детально фичу продукта. Для чего она? Что должна делать? Какой у нее приоритет выполнения?.. Из этого раздела читающему срс человеку должно сразу стать понятно зачем этот функционал присутствует в системе.

▪ **FRQ \ FRQ X \ Stimulus/Response sequences**

- Триггер запуска фичи. Когда она запускается и как себя ведет при запуске? Например, HTML редактор показывается при нажатии пользователя на ссылку меню Открыть HTML редактор

▪ **FRQ \ FRQ X \ Functional requirements**

- Подробное и детальное описание фичи. Описываем все: как работает, как реагирует на ошибки, что должно проверять, как отображать данные, как и куда что сохраняет и тд

НФТ

- **Определяют качественные характеристики системы** – то, как она должна выполнять свои функции. Если функциональные отвечают на вопрос «что делать?», то нефункциональные сосредоточены на вопросе «как хорошо это делать?». НФТ должны быть точно сформулированы и иметь чёткие критерии проверки (**количественное определение**) их достижимости.
- **Производительность** – время отклика, пропускная способность, использование ресурсов.
- **Надёжность** – устойчивость к сбоям, доступность, восстанавливаемость.
- **Безопасность** – конфиденциальность, целостность, аутентификация, авторизация.
- **Масштабируемость** – способность системы расти с увеличением нагрузки.
- **Удобство использования** – интуитивность, доступность, обучаемость.
- **Поддерживаемость** – модульность, тестируемость, адаптируемость.
- **Совместимость** – взаимодействие с другими системами, соответствие стандартам.

Пример формулировки: время отклика системы не должно превышать 200 мс при нагрузке до 1000 одновременных пользователей.

Структура. НФТ

- **Производительность** – время отклика, максимальная нагрузка, пропускная способность, использование ресурсов.
- **Безопасность** – шифрование, контроль доступа, аудит, конфиденциальность, целостность, аутентификация, авторизация.
- **Юзабилити** – адаптивность, доступность, понятность.
- **Надёжность** – резервное копирование, откат, устойчивость к сбоям, доступность, восстанавливаемость.
- **Масштабируемость** – рост нагрузки без деградации.
- **Совместимость** – работа на разных устройствах, браузерах, взаимодействие с другими системами, соответствие стандартам.
- **Поддерживаемость** – логирование, читаемый код, документация, модульность.
- **Локализация** – мультиязычность, форматы дат и валют.

Структура. Требования к внешним интерфейсам

- ✓ Для пользовательских интерфейсов
- ✓ Для графических интерфейсов
- ✓ Для аппаратных интерфейсов
- ✓ Для программных интерфейсов
- ✓ Для коммуникационных интерфейсов
- ✓ и др.

Структура. Ограничения и допущения

- Язык программирования, база данных
- Стандарты кодирования
- Стандарты обмена данными
- Ограничения накладываемые окружением, допустим совместимость только с ФФ
- Ограничения которые могут быть наложены бизнес-логикой проекта